

S.Y.B.SC.(DATA SCIENCE) SEM- III

Subject: Data Structure – I

Unit1 : Introduction to Data Structures and Algorithm Analysis

By,

Mrs. Reshma Masurekar

Dr. D. Y. Patil ACS College,

Pimpri, Pune 18.

Unit1 : Introduction to Data Structures and Algorithm Analysis

- Overview of Data Structure
 - Need of Data Structure,
 - Definitions: Data and information, Data type, Data object, ADT, Data Structure
 - Types of Data Structures: Linear vs NonLinear, Static vs. Dynamic
- Complexity Analysis
 - Space and time complexity
 - Asymptotic notations (Big O, Omega Ω , Theta Θ),
 - Best, Worst, Average case analysis

Algorithm analysis

- Analysis of algorithms is the determination of the amount of time and space resources required to execute it.
- Analysis of algorithm is the process of analyzing the problem-solving capability of the algorithm in terms of the time and size required.

Algorithm

- Algorithm is a *step-by-step procedure*, which defines a set of instructions to be executed in a *certain order* to get the desired output.
- **Input** – An algorithm should have 0 or more well-defined inputs.
- **Output** – An algorithm should have 1 or more well-defined outputs, and should match the desired output.
- **Finiteness** – Algorithms must terminate after a finite number of steps.
- **Definiteness** – Algorithm should be clear and unambiguous. Each of its steps (or phases), and their inputs/outputs should be clear and must lead to only one meaning.
- **Effectiveness** – Each instruction must be sufficiently basic, then it can be done effectively and in finite time by a person using only paper and pencil.

Types of Algorithm Analysis

- **Best case:** Define the input for which algorithm takes less time or minimum time. In the best case calculate the lower bound of an algorithm.

Example: In the linear search when search data is present at the first location of large data then the best case occurs.

- **Worst Case:** Define the input for which algorithm takes a long time or maximum time. In the worst calculate the upper bound of an algorithm.

Example: In the linear search when search data is not present at all then the worst case occurs.

- **Average case:** In the average case take all random inputs and calculate the computation time for all inputs. And then we divide it by the total number of inputs.

Average case = all random case time / total no of case

Performance analysis

- Performance analysis of an algorithm is the process of **calculating space and time required by that algorithm.**
- Performance analysis of an algorithm is performed by using following measures:
 - Space complexity
 - Time complexity

Space Complexity

- **Space complexity** of an algorithm represents the amount of memory space required by the algorithm in its life cycle.
- Memory is required to store program instructions, constant values, variable values.
- Space complexity of an algorithm is defined as “**total amount of computer memory required by an algorithm to complete its execution.**”

SUM(P, Q)

Step 1 - START

Step 2 - $R \leftarrow P + Q + 10$

Step 3 – Stop

Here we have three variables P, Q and R and one constant. Hence $S(p) = 1+3$. Now space is dependent on data types of given constant types and variables and it will be multiplied accordingly.

- Space needed by an algorithm is equal to the sum of the following two components:
 - A **fixed part** that is a space required **to store certain data and variables** (i.e. simple variables and constants, program size etc.), that are not dependent of the size of the problem.
 - A **variable part** is a space required by variables, whose **size is totally dependent on the size** of the problem. For example, recursion stack space, dynamic memory allocation etc.
- Space complexity $S(p)$ of any algorithm p is $S(p) = A + Sp(I)$ Where A is treated as the fixed part and $S(I)$ is treated as the variable part of the algorithm which depends on instance characteristic I .

Time complexity

- Time Complexity of an algorithm is the representation of the **amount of time required by the algorithm to execute to completion.**

Read a

Read b

$C=a+b$

Write c

Time complexity = 4 units.

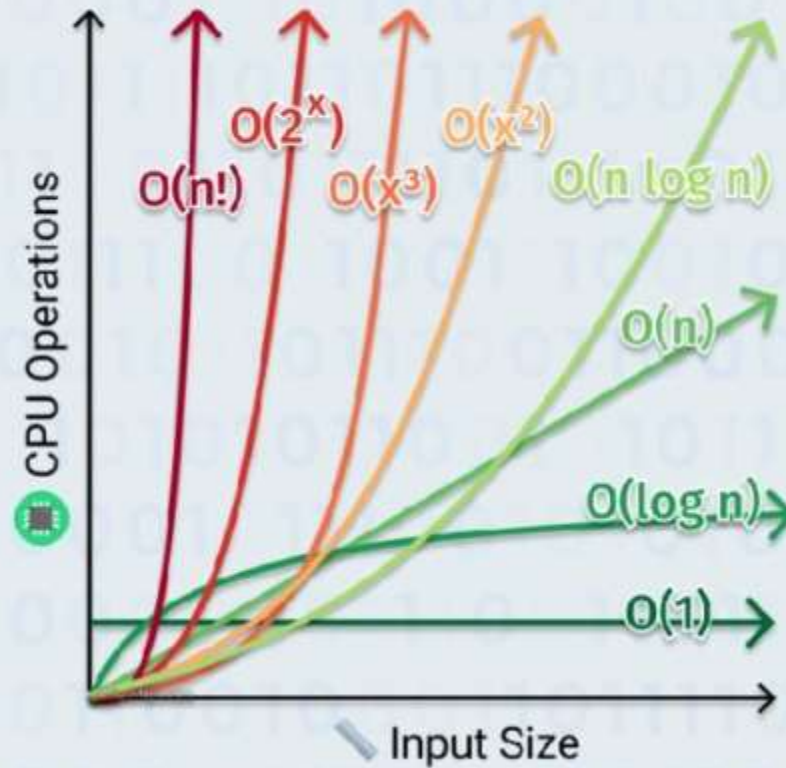
Consider 1 instruction requires 1 logical unit of time.

This algorithm is instance independent i.e. time is fixed.

Hence time complexity is $O(1)$.

Here $O(1)$ denotes constant time complexity.

🕒 Time Complexity



Asymptotic Notations

- Asymptotic notations are mathematical tools used to describe the performance of algorithms, particularly their growth rate, as the input size increases.
- Asymptotic Notation in Data Structure is a way to describe how the time or space needed by an algorithm grows as the size of the input grows. It helps understand how efficient an algorithm is when we have very large inputs.
- They allow us to compare different algorithms and understand their efficiency.
- Imagine you have different ways to solve a problem, like sorting a list of numbers. Some ways are fast when the list is short, but they might become very slow when the list is long. Asymptotic notation helps us compare these methods by focusing on how they perform when the input size gets really big.
- *In simple terms, asymptotic notation helps us predict how an algorithm will perform and make better choices when writing programs.*
- The main asymptotic notations are Big O, Big Omega, and Big Theta.

Big O (O)

- The notation Big O is the formal way to express the **upper bound of an algorithm's running time**.
- It is the most commonly used notation.
- It measures the **worst case time complexity** or the longest amount of time an algorithm could possibly take to complete.

- **Definition:** A function $f(n)$ can be represented is the order of $g(n)$ that is $O(g(n))$, if there exists a value of positive integer n as n_0 and a positive constant c such that –

$$f(n) \leq c \cdot g(n) \text{ for } n > n_0 \text{ in all case}$$

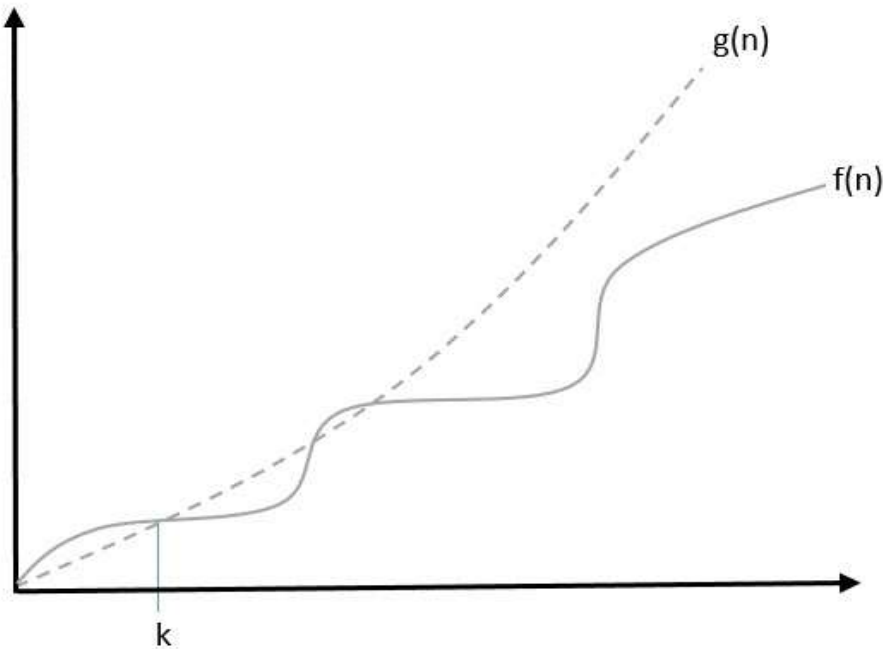
Hence, function $g(n)$ is an upper bound for function $f(n)$, as $g(n)$ grows faster than $f(n)$.

Example:

If an algorithm takes $O(n)$ time, it means its execution time grows linearly with the input size 'n'. A classic example is searching a list of size 'n' element by element.

Other examples:

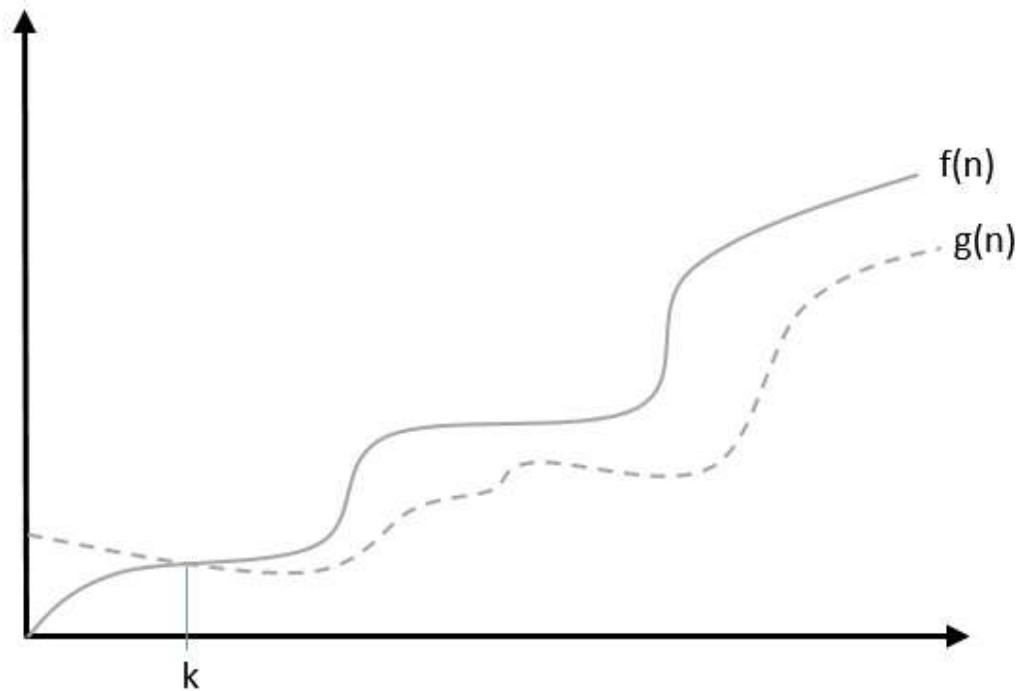
$O(n^2)$, $O(\log n)$, $O(2^n)$.



Big Omega (Ω)

- The notation $\Omega(n)$ is the formal way to express the **lower bound of an algorithm's running time**.
- It measures the **best case time complexity** or the best amount of time an algorithm can possibly take to complete.
- **Definition:** We say that $f(n)=\Omega(g(n))$ when there exists constant c that -
$$f(n) \geq c \cdot g(n)$$
 for all sufficiently large value of n . Here n is a positive integer. It means function g is a lower bound for function f ; after a certain value of n , f will never go below g .

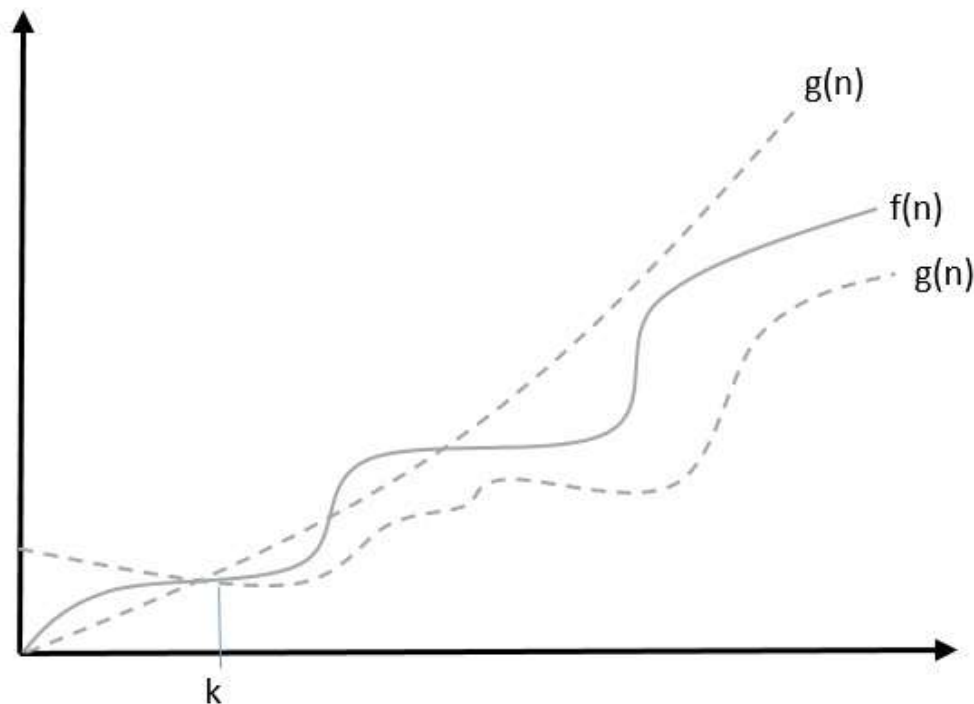
- **Example:** If an algorithm takes $\Omega(n)$ time, it means its execution time will always be at least proportional to 'n'.
- **How to find:** Identify the smallest possible execution time for a given input size.



Big Theta (Θ)

- The notation $\Theta(n)$ is the formal way to express **both the lower bound and the upper bound** of an algorithm's running time.
- Some may confuse the theta notation as the **average case time complexity**; while big theta notation could be *almost* accurately used to describe the average case, other notations could be used as well.
- We say that $f(n) = \Theta(g(n))$ when there exist constants c_1 and c_2 that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all sufficiently large value of n . Here n is a positive integer.
This means function g is a tight bound for function f .

- **Example:**
- If an algorithm takes $\Theta(n)$ time, it means its execution time grows linearly with the input size 'n', and it's both the best and worst case.
- **How to find:**
- Identify the dominating term in the algorithm's execution time and discard constant factors.



Importance of Asymptotic Notations

- **Algorithm Comparison:** They allow us to compare different algorithms and choose the most efficient one based on the expected input size.
- **Performance Prediction:** They help predict how the execution time of an algorithm will scale as the input grows.
- **Resource Allocation:** They inform resource allocation decisions, such as memory allocation or processing time, based on the algorithm's growth rate.

Applications of Algorithm Asymptotic Notation

- **Software Development**

Developers use asymptotic notation to choose the most efficient algorithms for their applications, ensuring that the software can handle large amounts of data effectively.

- **Data Analysis**

Data scientists analyze large datasets and need efficient algorithms to process data quickly. Asymptotic notation helps in selecting algorithms that can scale with the data.

- **Network Design**

Network engineers use algorithms to optimize routing and manage data flow. Understanding the efficiency of these algorithms ensures that the network can handle high traffic volumes.

- **Artificial Intelligence**

AI applications, such as machine learning and game playing, require efficient algorithms to process large amounts of data and make decisions in real-time.

- **Web Development**

Handling large user requests efficiently is critical for web applications.

Limitations of Asymptotic Notation

- **Ignores Constant Factors:**

Asymptotic notation **focuses on the growth rate** and ignores constant factors. In practice, these constants can significantly affect performance for small input sizes.

Example: An algorithm with **time complexity** $O(n)$ might be faster than an $O(1)$ algorithm for small inputs due to lower constant overhead.

- **Does Not Account for Lower-order Terms:**

It disregards lower-order terms that can impact performance for specific input sizes.

Example: $O(n + \log n)$ is simplified to $O(n)$, but for small values of n , the $\log n$ term might still be significant.

- **Assumes Infinite Input Size:**

Asymptotic notation is most useful for large inputs and may not provide accurate insights for small or moderate input sizes.

Example: An $O(n^2)$ algorithm might outperform an $O(n \log n)$ algorithm for small datasets due to its simpler implementation.

- **Hardware and Implementation Details:**

It does not consider the impact of hardware, compiler optimizations, and specific implementation details that can affect the actual running time.

Example: An $O(n \log n)$ algorithm might perform worse than an $O(n^2)$ algorithm on a specific hardware due to cache misses or other low-level factors.

- **Average vs. Worst Case:**

It primarily focuses on worst-case scenarios (Big O), which might not always represent the typical performance of an algorithm.

Example: Quick Sort has an average-case time complexity of $O(n \log n)$, but its worst-case is $O(n^2)$. In practice, its average-case performance is more relevant for many applications.

Common Asymptotic Notations

Notation	Description	Example
$O(1)$	Constant time	Accessing an array element
$O(\log n)$	Logarithmic time	Binary search
$O(n)$	Linear time	Finding max in an array
$O(n \log n)$	Linearithmic time	Merge sort
$O(n^2)$	Quadratic time	Bubble sort
$O(2^n)$	Exponential time	Recursive Fibonacci
$O(n!)$	Factorial time	Generating permutations

Apriori and Apostiari Analysis

- **Apriori analysis** means, analysis is performed prior to running it on a specific system. This analysis is a stage where a function is defined using some theoretical model. Hence, we determine the time and space complexity of an algorithm by just looking at the algorithm rather than running it on a particular system with a different memory, processor, and compiler.
- **Apostiari analysis** of an algorithm means we perform analysis of an algorithm is performed only after running it on a system. It directly depends on the system and changes from system to system.